

CONTEXTO

Aplicações desenvolvidas com Ionic e Angular utilizam o Capacitor para acessar recursos nativos dos dispositivos móveis, como câmera, sistema de arquivos, notificações e geolocalização. O Capacitor atua como uma camada de integração entre a aplicação web e as APIs nativas do Android.

Neste roteiro será utilizado o plugin Camera do Capacitor para capturar imagens diretamente pela câmera do dispositivo móvel. Além disso, serão utilizados recursos para armazenamento local das imagens e persistência de dados da aplicação.

Todas as informações e exemplos apresentados neste guia foram extraídos e adaptados do tutorial "*Your First Ionic App: Angular*", disponível na documentação oficial do Ionic: <https://ionicframework.com/docs/angular/your-first-app>

DESENVOLVIMENTO DO PROJETO

Criar o projeto

- ionic start MinhasFotos
 - Angular
 - blank
 - Standalone

Criar uma página, uma classe e um serviço

- ionic g page pages/inicio
- ionic g class model/arquivo.model
- ionic g service services/arquivo.service

Executar os comandos na pasta do projeto

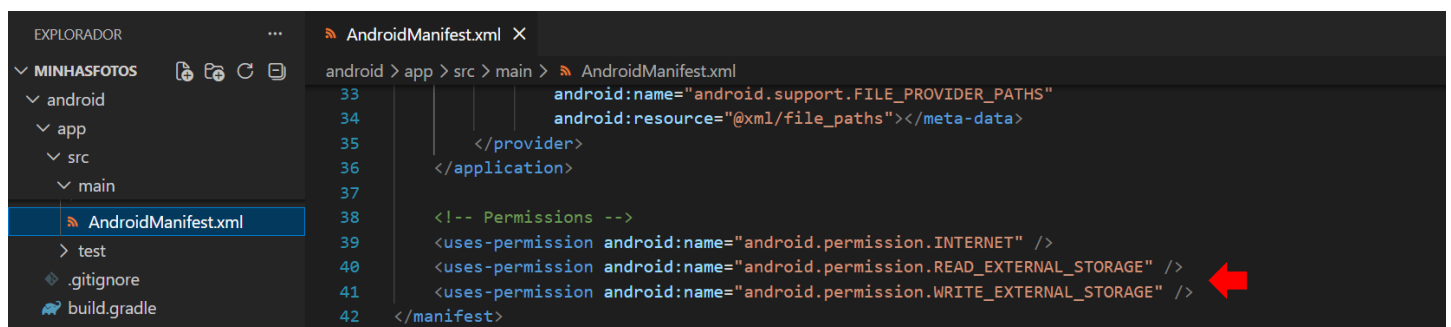
- ionic capacitor add android
 - Adiciona a plataforma Android ao projeto Ionic, criando a estrutura necessária para geração e execução do aplicativo em dispositivos Android.
- npm install -g @ionic/cli native-run cordova-res
 - Instala ferramentas auxiliares utilizadas durante o desenvolvimento. O Ionic CLI permite gerenciar o projeto, o Native Run facilita a execução em dispositivos físicos e o Cordova Res auxilia na geração de ícones e telas de abertura.
- npm install @capacitor/camera @capacitor/preferences @capacitor/filesystem
 - Instala os plugins do Capacitor responsáveis pelo acesso à câmera do dispositivo, armazenamento de preferências e manipulação de arquivos locais.
- npm install @ionic/pwa-elements
 - Instala componentes que permitem simular funcionalidades nativas, como o acesso à câmera, quando a aplicação é executada diretamente no navegador.
- npx cap sync
 - Sincroniza as dependências e configurações do Capacitor com a plataforma Android, atualizando o projeto nativo após a instalação ou alteração de plugins.

Abrir e executar o projeto

- ionic serve

Ajustar as configurações do projeto

- Alterar o arquivo [AndroidManifest.xml](#)
 - [android/app/src/main/AndroidManifest.xml](#)



- Código
 - <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
 - <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

- Alterar o arquivo `main.ts`
 - `src/main.ts`

```

1 import { bootstrapApplication } from '@angular/platform-browser';
2 import { RouteReuseStrategy, provideRouter, withPreloading, PreloadAllModules } from '@angular/router';
3 import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular/standalone';
4 import { defineCustomElements } from '@ionic/pwa-elements/loader';
5
6 import { routes } from '../app/app.routes';
7 import { AppComponent } from '../app/app.component';
8
9 bootstrapApplication(AppComponent, {
10   providers: [
11     { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
12     provideIonicAngular(),
13     provideRouter(routes, withPreloading(PreloadAllModules)),
14   ],
15 });
16
17 defineCustomElements(window);

```

Ajustar a estrutura do projeto

- excluir a pasta "**home**" e seus arquivos, realizando também os ajustes no arquivo `app-routing.module.ts`
- definir o redirecionamento para a rota "**inicio**" e excluir a rota "**home**"

```

1 import { Routes } from '@angular/router';
2
3 export const routes: Routes = [
4   {
5     path: '',
6     redirectTo: 'inicio',
7     pathMatch: 'full',
8   },
9   {
10    path: 'inicio',
11    loadComponent: () => import('../pages/inicio/inicio.page').then( m => m.InicioPage)
12  },
13 ];

```

Implementar o código da classe arquivo

- O código do arquivo `arquivo.model.ts` após as implementações é ilustrado abaixo:

```

1 export class ArquivoModel {
2   id: number;
3   imagem: string;
4
5   constructor(){
6     this.id = 0;
7     this.imagem = "";
8   }
9 }

```

Implementar o código do serviço arquivo

- O código do arquivo `arquivo.services.ts` após as implementações é ilustrado abaixo:
 - A função `capturar()` é responsável por acionar a câmera do dispositivo, capturar uma fotografia e armazenar a imagem.
 - O método `Camera.getPhoto()` do plugin `Camera` do Capacitor é executado de forma assíncrona utilizando a palavra-chave `await`. Esse método abre o aplicativo de câmera nativo do dispositivo e aguarda a ação do usuário. Os parâmetros utilizados são:
 - **resultType: CameraResultType.Base64:** Define que a imagem será retornada como uma `string` codificada em `Base64`.
 - **source: CameraSource.Camera:** Indica que a captura deve ser realizada utilizando a câmera do dispositivo.
 - **quality: 50:** Define a qualidade da imagem gerada em uma escala de 0 a 100.
 - Após a captura da foto, o conteúdo da imagem é disponibilizado na propriedade `base64String` do objeto retornado pelo plugin. Esse valor é então atribuído ao atributo `imagem` do objeto `arquivo`.

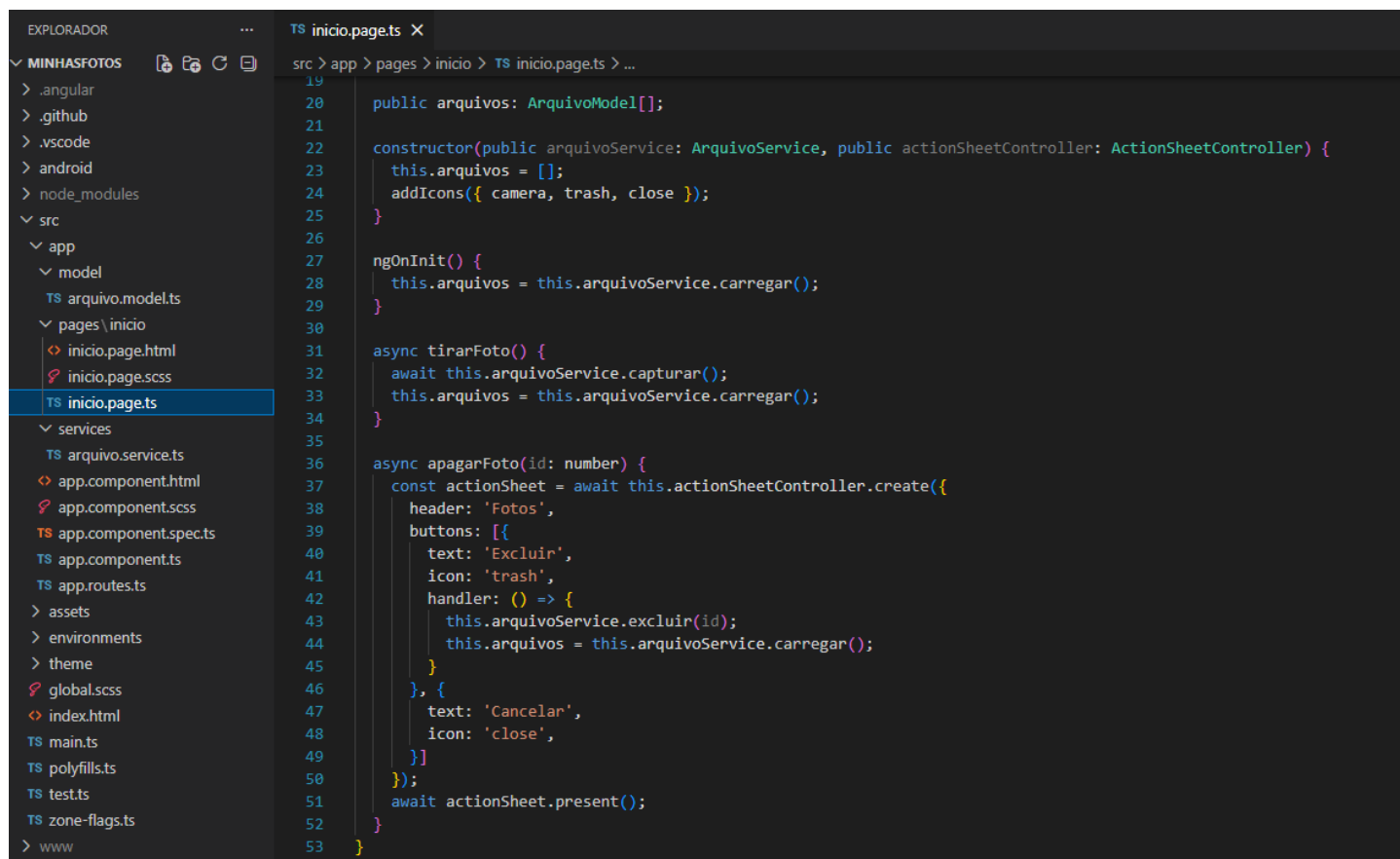
- O **operador !** informa ao compilador TypeScript que a propriedade não será nula naquele contexto.
- Toda a operação é envolvida por um bloco **try/catch**. Caso o usuário cancele a captura da foto ou ocorra algum erro durante a execução, a exceção é tratada e uma mensagem é exibida no console.
- Os demais métodos são responsáveis pelo gerenciamento dos dados no localStorage. O método `armazenar()` realiza a inclusão ou atualização de arquivos, o método `excluir()` remove um arquivo a partir do identificador informado e o método `carregar()` recupera a lista de arquivos armazenados, retornando uma coleção vazia caso não existam registros salvos.
- Observações:
 - **Base64** é um formato de codificação que converte dados binários, como imagens, arquivos e vídeos, em uma sequência de caracteres de texto. Essa abordagem facilita o armazenamento e a transmissão desses dados entre aplicações e APIs. Por exemplo: `iVBORwOKGgoAAAANSUhEUgAA...`
 - Embora **CameraResultType** e **CameraSource** estejam marcados como preteridos (*deprecated*), eles continuam funcionando normalmente e foram utilizados neste exemplo por simplicidade e compatibilidade com versões atuais do Capacitor.

```
TS arquivo.service.ts X
src > app > services > TS arquivo.service.ts > ArquivoService > armazenar
1  import { Injectable } from '@angular/core';
2  import { Camera, CameraResultType, CameraSource } from '@capacitor/camera';
3  import { ArquivoModel } from '../model/arquivo.model';
4
5  @Injectable({
6    providedIn: 'root',
7  })
8  export class ArquivoService {
9
10   constructor() {
11   }
12
13   public async capturar() {
14     try {
15       let arquivo = new ArquivoModel();
16       const camera = await Camera.getPhoto({
17         resultType: CameraResultType.Base64,
18         source: CameraSource.Camera,
19         quality: 50
20       });
21
22       arquivo.imagem = camera.base64String!;
23       this.armazenar(arquivo);
24
25     } catch (erro) {
26       console.log('Captura cancelada pelo usuário.');

```

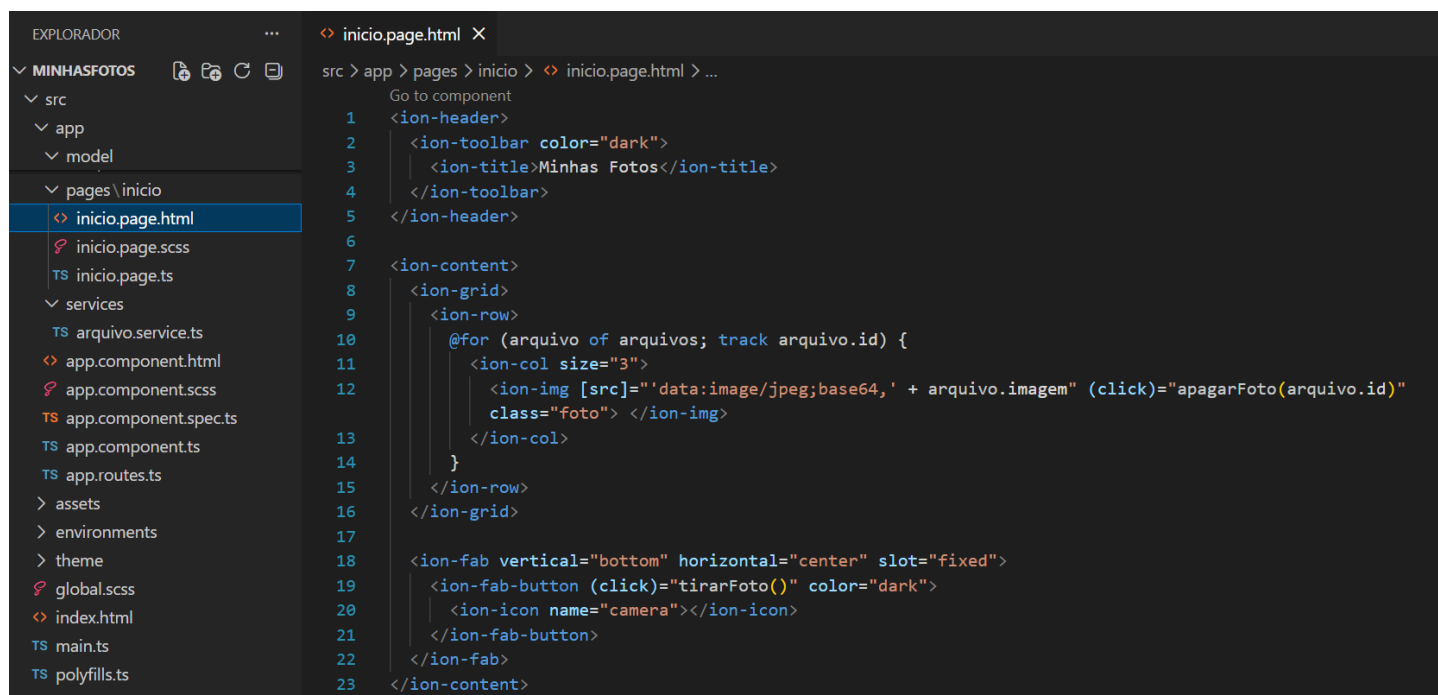
Implementar os códigos dos arquivos da [página início](#)

- Código do arquivo [início.page.ts](#) após as implementações é ilustrado abaixo:
 - A classe **InicioPage** concentra as funcionalidades de captura, exibição e exclusão de imagens. A interação com os dados é realizada por meio do **ArquivoService**, responsável pelo acesso ao **localStorage**, enquanto o **ActionSheetController** é utilizado para apresentar um menu de confirmação ao usuário antes da remoção de uma imagem.



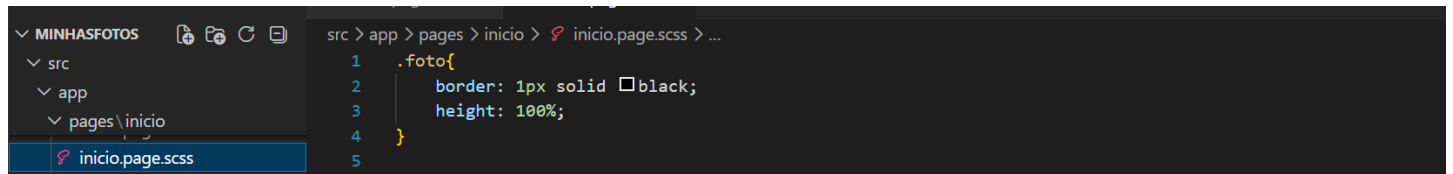
```
19 public arquivos: ArquivoModel[];
20
21
22 constructor(public arquivoService: ArquivoService, public actionSheetController: ActionSheetController) {
23   this.arquivos = [];
24   addIcons({ camera, trash, close });
25 }
26
27 ngOnInit() {
28   this.arquivos = this.arquivoService.carregar();
29 }
30
31 async tirarFoto() {
32   await this.arquivoService.capturar();
33   this.arquivos = this.arquivoService.carregar();
34 }
35
36 async apagarFoto(id: number) {
37   const actionSheet = await this.actionSheetController.create({
38     header: 'Fotos',
39     buttons: [
40       {
41         text: 'Excluir',
42         icon: 'trash',
43         handler: () => {
44           this.arquivoService.excluir(id);
45           this.arquivos = this.arquivoService.carregar();
46         }
47       }, {
48         text: 'Cancelar',
49         icon: 'close',
50       }
51     ]
52   });
53   await actionSheet.present();
54 }
```

- Código do arquivo [início.page.html](#) após as implementações é ilustrado abaixo:
 - O prefixo `data:image/jpeg;base64` (linha 12), informa que o conteúdo fornecido está codificado em **Base64** e representa uma imagem no formato **JPEG**. Dessa forma, o valor armazenado em `arquivo.imagem` pode ser exibido diretamente no atributo `src` do componente `<ion-img>`, sem a necessidade de criar ou acessar um arquivo físico no dispositivo.



```
1 <ion-header>
2   <ion-toolbar color="dark">
3     <ion-title>Minhas Fotos</ion-title>
4   </ion-toolbar>
5 </ion-header>
6
7 <ion-content>
8   <ion-grid>
9     <ion-row>
10      @for (arquivo of arquivos; track arquivo.id) {
11        <ion-col size="3">
12          <ion-img [src]="data:image/jpeg;base64, ' + arquivo.imagem" (click)="apagarFoto(arquivo.id)"
13            class="foto"> </ion-img>
14        </ion-col>
15      }
16    </ion-row>
17  </ion-grid>
18
19  <ion-fab vertical="bottom" horizontal="center" slot="fixed">
20    <ion-fab-button (click)="tirarFoto()" color="dark">
21      <ion-icon name="camera"></ion-icon>
22    </ion-fab-button>
23  </ion-fab>
24 </ion-content>
```

- Código do arquivo [inicio.page.scss](#) após as implementações é ilustrado abaixo:



```
src > app > pages > inicio > inicio.page.scss > ...
1  .foto{
2      border: 1px solid black;
3      height: 100%;
4  }
5
```